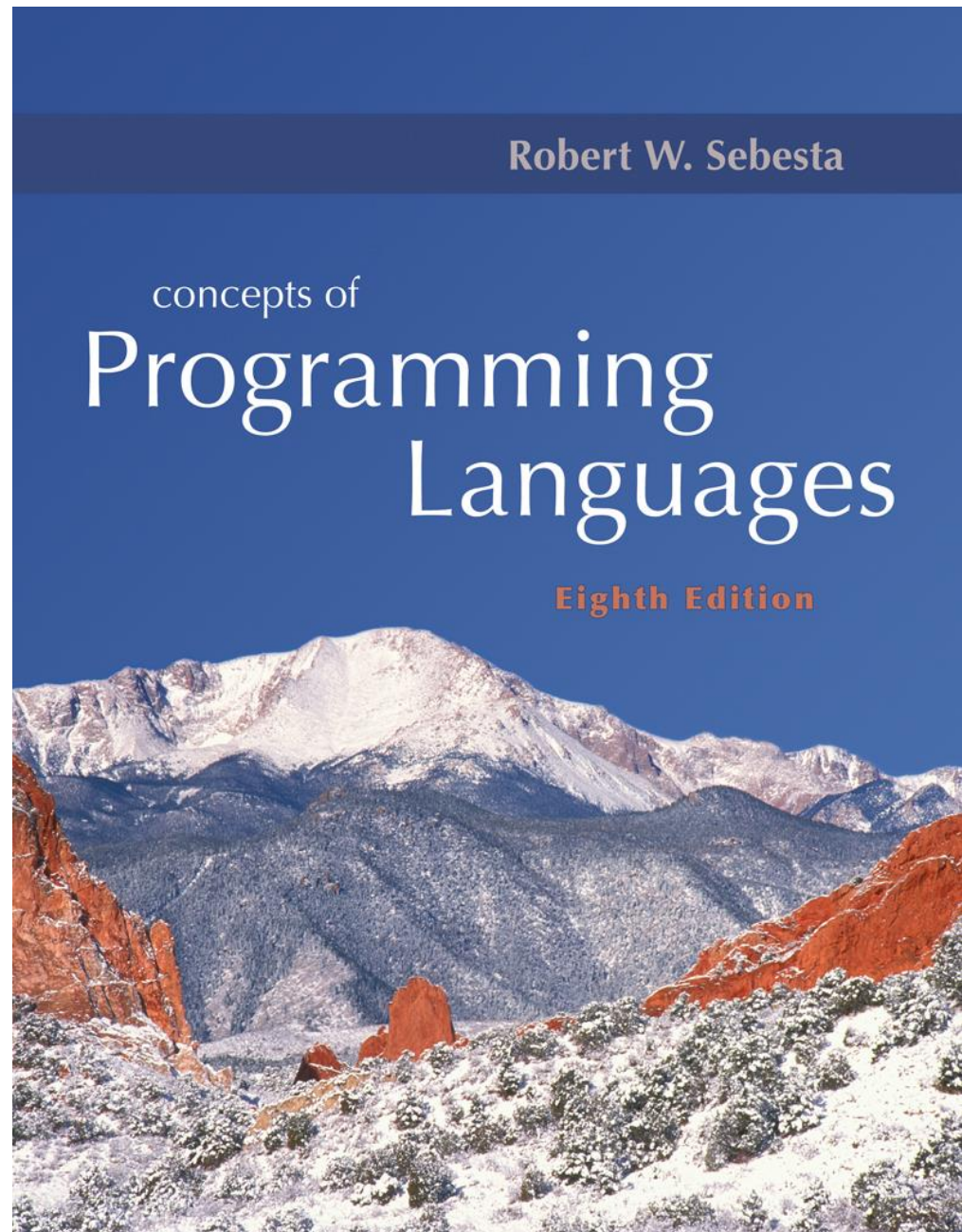


Chapter 9

Subprograms



ISBN 0-321-49362-1

Introduction

- Two fundamental abstraction facilities
 - Process abstraction
 - Emphasized from early days
 - Data abstraction
 - Emphasized in the 1980s

Fundamentals of Subprograms

- Each subprogram has a single entry point
- The calling program is suspended during execution of the called subprogram
- Control always returns to the caller when the called subprogram's execution terminates

Basic Definitions

- A *subprogram definition* describes the interface to and the actions of the subprogram abstraction
- A *subprogram call* is an explicit request that the subprogram be executed
- A *subprogram header* is the first part of the definition, including the name, the kind of subprogram, and the formal parameters
- The *parameter profile* (*signature*) of a subprogram is the number, order, and types of its parameters

Basic Definitions (Cont')

- Function declarations in C and C++ are often called *prototypes*
 - *void WriteMyString();*
 - *char *test (int);*
 - *int check(int a, double b);*
- A *subprogram declaration* provides the protocol, but not the body, of the subprogram
- There are two ways that subprogram can gain access to the data: through *direct access to nonlocal variables* or through *parameter passing*
- A *formal parameter* are listed in the subprogram header and used in the subprogram
- An *actual parameter* represents a value or address used in the subprogram call statement

Actual/Formal Parameter Correspondence – Positional

- The binding of actual parameters to formal parameters is by position: the first actual parameter is bound to the first formal parameter and so forth
- Safe and effective
- When lists are long, it is easy for a programmer to make mistakes in the order of actual parameters in the list
- Most of programming languages

Actual/Formal Parameter Correspondence – Keyword

- The name of the formal parameter to which an actual parameter is to be bound is specified with the actual parameter
- Parameters can appear in any order
- How to call?

```
sumer(length = 20, list = my_array, sum = 10)
```

- The user of the subprogram should know the names of formal parameters

Actual/Formal Parameter Correspondence – Keyword (Cont')

- In addition to keyword parameters, Ada, Fortran 95+ and Python allow positional parameters. Keyword and positional parameters can be mixed in a **call**, as in
 - *sumer(my_length, sum = 10, list = myList)*
- The only restriction with this approach is that after a keyword parameter appears in the list, all remaining parameters must be keyworded

Formal Parameter Default Values

- In certain languages (e.g., C++, Ada), formal parameters can have default values (if not actual parameter is passed)
 - In C++, default parameters must appear last because parameters are **positionally associated**
 - *float compute(float income, float tax_rate, int x= 1)*
- Consider the following *Python* function header:
 - *def compute(income, x = 1, tax_rate):*
 - *Using print function in Python*
 - *print (x,y, sep==">")*

Given the following Python head function

```
def fun1(x, sin=0, cos=PI):
```

which from the following are valid call?

- fun1(3) ✓
- fun1(x=3) ✓
- fun1(3,2.4,0) ✓
- fun1(3,2.4,sin=0) ✗
- fun1(sin=3,x=2.4,0) ✗
- fun1(cos=3,sin=2.4, sin=0) ✗
- fun1(cos=3,sin=1, x=0) ✓

Variable Number of Parameters

- C# methods can accept a variable number of parameters as long as they are of the same type

```
public void DisplayList(params int[] list) {  
    foreach (int next in list) {  
        Console.WriteLine("value {0}", next);  
    }  
}
```

C# Example

```
class C1 {  
    private static void Average(string Text, params int[] Arr) {  
        int sum = 0;  
        System.Console.Write("Average of {0} (", Text);  
        for (int i = 0; i < Arr.Length; i++) {  
            sum += Arr[i];  
            System.Console.Write(Arr[i] + ", ");  
        }  
        System.Console.WriteLine("): {0}", (float)sum / Arr.Length);  
    }  
    static void Main() {  
        Average ("First List: ", 3, 6, 3);  
        Average ("Second List: ", 4, 4, 6, 6, 5);  
    }  
}
```

Procedures and Functions

- There are two categories of subprograms
 - *Procedures* are collection of statements that define parameterized computations
 - *Functions* structurally resemble procedures but are semantically modeled on mathematical functions
 - They are expected to produce no side effects
 - In practice, program functions have side effects

Design Issues for Subprograms

- What parameter passing methods are provided?
- Are parameter types checked?
- Are local variables static or dynamic?
- Can subprogram definitions appear in other subprogram definitions?
- Can subprograms be overloaded?
- Can subprogram be generic?